

La tienda toca el mundo

useEffect: persistencia, dependencias y limpieza. El laboratorio del cronómetro. Y el salto grande: la API real con su adaptador.

4 h 15 · L-M-V

Retomamos donde cortamos

STEERCL

SIN REPASOS LARGOS: RETOMAMOS Y SEGUIMOS

El plan de hoy

0. Reapertura

10'

el punto de corte + tareas destacadas

B. useEffect

~105'

persistencia · dependencias · limpieza · StrictMode

C. La API real

~90'

el adaptador · fetch en un efecto · estados honestos

A. Remate: carrito y filtros

0-90'

solo lo pendiente · guion de la clase 3

L. Laboratorio: el cronómetro

~45'

fuera del proyecto · 4 actos, 4 roturas

F. Cierre + 5 tareas

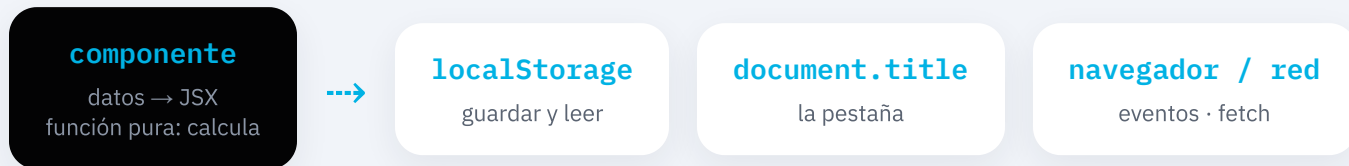
25'

push demostrado, dudas y anticipo

**Receso de 15 min** alrededor de las 2:00 · verde = **el corazón del día** · se corta a las 3:50 donde toque

F5 Y EL CARRITO MUERE · QUÉ ES UN EFECTO COLATERAL

El mundo **fuera del render**



Eso de la derecha es **tocar el mundo**: efectos colaterales. La regla de React: **el render calcula, no toca.**

Para tocar existe una pieza aparte, el segundo hook del curso: `useEffect`.

Anatomía de `useEffect`

```
useEffect(() => {  
  // QUÉ hacer: tocar el mundo  
  localStorage.setItem(CLAVE, JSON.stringify(carrito));  
}, [carrito]); // CUÁNDO: si esto cambió desde el render anterior
```

1 · render

la función corre



2 · pinta

la pantalla primero



3 · efecto

el mundo después

Es un hook: arriba del componente, dentro de la función, nunca en un `if`.

EL ARRAY DE DEPENDENCIAS

Las tres formas del array

sin array

corre tras **cada render**. Casi nunca lo quieres.

[]

corre **una vez, al montar**. Para lo que se hace al arrancar.

[carrito]

reacciona: corre cuando ese valor cambió.

Dependencias honestas: todo valor del render que el efecto usa, va en el array. Mentirle deja efectos congelados en el pasado, y ese bug muerde semanas después.

LA PROMESA MÁS VIEJA DEL BLOQUE, PAGADA

StrictMode y la limpieza

En desarrollo, StrictMode **monta** → **desmonta** → **monta**. Es un simulacro: si tu efecto no soporta correr dos veces, está mal cerrado, y te lo hace notar HOY, no en producción.

escuchando / limpiado / escuchando

con limpieza: cada encendido tiene su apagado. Queda UNA suscripción viva.

escuchando / escuchando

sin limpieza: dos suscripciones, la primera huérfana para siempre.

Efecto que enciende, limpieza que apaga — y la traza balanceada es tu prueba.

LA DOCTRINA, CON SUS DOS MITADES

El anti-efecto

```
const [total, setTotal] = useState(0);  
useEffect(() => { setTotal(carrito.reduce(...)); }, [carrito]);  
  
const total = carrito.reduce((s, i) => s + i.precio * i.cantidad, 0);
```

Un efecto que calcula desde el estado: funciona, y son dos renders donde bastaba uno, un estado redundante y dos fuentes de verdad.

Si se puede calcular, es derivado: ni estado ni efecto. El efecto es solo para tocar el mundo.

LA API REAL · TODO DECIDIDO CON LAS REGLAS DEL CURSO

¿Dónde vive **un fetch**?

los productos que llegan

¿se pueden calcular? No: vienen de afuera → **estado**

el acto de pedirlos

¿toca el mundo? Sí: red → **efecto**, al montar: []

la forma inglesa de la API

title, price, thumbnail → no sale de api.ts: el **adaptador** traduce al contrato

si la red falla

el catálogo semilla responde y la tienda **avisa**: degradación elegante

```
useEffect(async () => { ... }, []) // Promise no es limpieza: no compila  
// el patrón: función async adentro, y llamarla
```


HASTA DONDE LLEGAMOS HOY

Lo de hoy

efecto colateral

tocar el mundo (disco, título, red). El render calcula; el efecto toca

dependencias

sin array · [] · [x] — y honestas: lo que usas, lo declaras

persistencia

guardar con [carrito] + leer al arrancar con aduana. El F5, vencido

limpieza

efecto que enciende, limpieza que apaga — StrictMode caza al que no

adaptador

la forma cruda no sale de api.ts; el contrato manda en la tienda

estados honestos

EstadoCarga por fin dice la verdad: cargando, listo, respaldo

CRITERIO DE ENTREGA: NPM RUN BUILD SIN ERRORES + PUSH

5 tareas

FÁCIL

Replica la clase, hasta donde llegamos, con la grabación al lado. Es LA tarea.

INTERMEDIO

Ejercicio 1: persistir carritoAbierto con su aduana (typeof para el booleano).

INTERMEDIO

Ejercicio 2: el título completo de la pestaña, con las dependencias bien declaradas.

DIFÍCIL

Ejercicio 4: un campo nuevo de la API de punta a punta: contrato → adaptador → tarjeta.

DIFÍCIL

Ejercicio 3 + la pregunta: auditoría del anti-efecto en tu App, y tu hipótesis: ¿qué "limpia" la limpieza de un fetch?

Extra del laboratorio: **la cuenta regresiva** (arranca en 60, se detiene sola en 0, con pausar y reiniciar).

El cronómetro, en 4 actos

1 · se congela en 1

el intervalo vive en la primera foto → la **forma funcional es obligatoria**: $(s) \Rightarrow s + 1$

2 · avanza de 2 en 2

StrictMode montó doble y nadie limpió → `clearInterval(id)` en el return

3 · ocultar = desmontar

la limpieza corre, y al volver arranca en 0: el estado muere con la instancia

4 · pausar y reanudar

la limpieza también corre **antes de cada re-ejecución** del efecto (deps: `[corriendo]`)

Próxima clase — retomamos donde cortamos

¿Qué limpia la limpieza de un fetch?

```
// mira la pestaña Network con atención:
```

```
GET /products x2 ← ¿por qué DOBLE?
```

```
const controlador = new AbortController()
```

En el camino: la respuesta a la pregunta de la difícil, el **skeleton** para que la carga se vea de tienda grande, y el salto de página única a aplicación: **rutas**, el detalle con URL propia y una 404 nuestra.

Trae las tareas hechas y el build en cero. Nos vemos.